TECHNICAL TEST REPORT

EDR evasion via direct syscalls: diagnosis, control attribution, then three bypass phases culminating in a successful syscall execution with the Sophos agent active

Defense layer	Verdict
SophosED user-mode hooks	BYPASSED
HitmanPro.Alert anti-exploit (SysCallX64)	BYPASSED
Alerts raised in the Sophos console	ZERO
Syscall executed with the agent active	YES (288 KB)

Author	0xKuraishY
Test dates	September 16-17, 2026
Tool under test	ActiveBreach-Engine v0.2.0 (open source)
Target	Windows 11 Pro 25H2 - build 26200.9445 (Proxmox VM)
EDR under test	Sophos Endpoint: SophosED + HitmanPro.Alert (Intercept X 2024.1.2.10)
Distribution	Personal research - free document

All tests were conducted on an isolated lab VM with a prior snapshot. This document describes offensive/defensive security research carried out in a controlled environment: the goal is to measure what the EDR actually observes and to derive detection recommendations.

Executive summary

This report documents the full evaluation of ActiveBreach Engine (ABE), an open-source direct syscall execution tool, against Sophos Endpoint deployed on a Windows 11 host. The test covers the initial diagnosis, attribution of the EDR controls, then three bypass phases culminating in a successful syscall execution with the agent active.

Key result

Both user-mode defense layers of Sophos (SophosED user-mode hooks and the HitmanPro.Alert SysCallX64 anti-exploit mitigation) were bypassed with minor modifications to a public tool. The complete sequence (bypassed initialization, executed syscall, process exit) produced no alert in the Sophos management console. Only the kernel layer observed the activity, without correlating it into a visible alert.

Defensive consequence

Detection of evasion tools cannot rely on user-mode hooks and heuristics. It must be grounded in kernel telemetry (stack-walking within callbacks, ETW Threat Intelligence) correlated into visible alerts, complemented by hardware-based prevention (HVCI / VBS). Details in section 6.

Table of contents

Executive summary 2

Key result . . . 2

Defensive consequence . . . 2

Table of contents 3

1. Context and objective 5

Test scope . . . 5

2. Test environment 5

3. Test procedure and diagnosis (phase 1) 6

3.1 Compiling ABE and building the PoC . . . 6

3.2 Initial symptom: infinite hang . . . 6

3.3 Dynamic analysis (System Informer) . . . 6

3.4 The debug build reveals the hidden error . . . 6

3.5 Analyzing the ABE source code . . . 7

3.6 Dumping the ntdll stubs . . . 7

3.7 Tracing the hook chain to SophosED.dll . . . 7

No Sophos alert: expected behavior . . . 8

3.8 A/B test: uninstalling Sophos . . . 9

4. Root cause: the complete mechanism 9

Precise formulation . . . 9

5. Conclusion 10

Update after the bypass phases (annexes C to E) . . . 10

6. Recommendations (detection and hardening) 10

6.1 Detecting such an evasion (independent of user-mode hooks) . . . 10

6.2 Prevention . . . 10

Governance . . . 10

Annex A - Key commands 11

Annex B - Excerpts: PoC and PowerShell scripts 11

Annex C - Evasion scenarios (phase 2) 12

C.1 Approach A (selected): patch ABE, VirtualAlloc fallback . . . 12

C.2 Approach B: homemade stub from the clean ntdll copy . . . 12

C.3 Approach C: reusing Sophos's relocated stub (read-only) . . . 12

C.4 Measurements at every step . . . 12

Annex D - Blocked by the anti-exploit 12

D.1 vm.rs patch v2: the neighboring stub trap . . . 12
D.2 Result under Sophos: initialization bypassed . . . 12
D.3 The next blocker: execution killed by the anti-exploit . . . 12
D.4 Why HMPA sees the syscall (SysCallX64 policy analysis) . . . 13
D.5 Intermediate summary . . . 13
Annex E - Complete bypass validated under active Sophos (phase 3) 14
E.1 The complete bypass chain (3 modifications) . . . 14
E.2 Result: syscall executed with Sophos active . . . 14
E.3 What this demonstration establishes . . . 15
Rigor note . . . 15
E.4 Updated recommendations . . . 15
Final finding . . . 15
References 15

1. Context and objective

ActiveBreach Engine (ABE) is an open-source tool (GitHub repository 8damon/ActiveBreach-Engine) that executes Windows system calls directly from user mode, without going through the standard API functions (kernel32/kernelbase), with the goal of evading the surveillance that EDRs place in the normal call path. The tool extracts syscall numbers (SSNs) from a clean copy of ntdll.dll, builds its own syscall stubs in memory, and invokes the kernel via the syscall instruction.

Three questions structure the test:

- Does the tool work as-is on a host protected by Sophos Endpoint?
- If not, why: active detection by Sophos or another cause?
- What should be retained for detection and fleet hardening?

Test scope

Isolated lab VM (prior snapshot), EDR deployed within this scope. The bypass phases (annexes C to E) were conducted in the same controlled environment.

2. Test environment

Hypervisor	Proxmox - isolated lab VM (snapshot taken before the tests)
OS	Windows 11 Pro 25H2 - build 26200.9445 (x64)
Toolchain	Rust 1.82 (x86_64-pc-windows-msvc), Visual Studio Build Tools 2022
Tool under test	ActiveBreach-Engine v0.2.0 - Rust SDK, ntdll_backend then secure backends
EDR under test	Sophos Endpoint: SophosED.dll + HitmanPro.Alert (Intercept X 2024.1.2.10, Core Agent 2026.2.1.3.0)
Test program	Custom Rust PoC (abe_poc) loading activebreach.dll via libloading
Analysis tools	System Informer, Event Viewer, PowerShell scripts (ReadProcessMemory)

3. Test procedure and diagnosis (phase 1)

3.1 Compiling ABE and building the PoC

ABE compiles without errors (two minor warnings). The PoC loads the DLL, calls activebreach_launch() then performs a read-only call: NtQuerySystemInformation (process list) via ab_call. Two pitfalls in the companion guide are worth noting: the Rust generics were mangled in the PDF (angle brackets interpreted as markup), and the guide claims launch() returns 0 = success, which is false in release builds (demonstrated in 3.4).

3.2 Initial symptom: infinite hang

First execution (release build): the process prints "activebreach_launch() returns: 0" then hangs indefinitely on the syscall call, with no message and 0% CPU. The guide interpreted the 0 as success: that is the first misleading clue of the sequence.

3.3 Dynamic analysis (System Informer)

- A single active thread (main thread), waiting in ntdll!WaitOnAddress, called from activebreach.dll!ab_call.
- No dispatcher thread starting in activebreach.dll, although the ABE mechanism requires one to execute syscalls.
- Three SophosED.dll threads injected into the process (permanent instrumentation).

Main thread stack at the time of the hang:

#	Frame	Detail
0	ntdll.dll!NtWaitForAlertByThreadId+0x14	internal ABE wait
1	ntdll.dll!RtlWaitOnAddress+0x213	(closest symbol)
2	KernelBase.dll!WaitOnAddress+0x38	
3	activebreach.dll!ab_violation_count+0x3f8	<- the ab_call invocation
4-5	activebreach.dll!ab_call+0x68	

The main thread is correctly waiting for a response from the dispatcher, which does not exist. The hang is a symptom, not the cause.

3.4 The debug build reveals the hidden error

Two discoveries in the ABE source code (src/internal/diagnostics.rs): in release mode, the error function returns 0 for every error; the 0 code therefore never indicates success in release.

```
pub fn AbErr(kind: ABError) -> u32 {
#[cfg(debug_assertions)]
{ *ERROR_CODES.lock().unwrap().get(&kind).unwrap() }
#[cfg(not(debug_assertions))]
{ let _ = kind; 0 } // <- IN RELEASE: EVERY ERROR IS RETURNED AS 0
}
```

Rebuild in debug: launch() returns -1076909169 (0xBFCFAB8F as u32) and the last log line is:

```
[*] Launching ActiveBreach...
[AB:MAPPER][TID:2852] Opening C:\Windows\System32\ntdll.dll
[+] activebreach_launch() returns: -1076909169
[*] Calling NtQuerySystemInformation via ABE... <- hangs here (ab_call)
```

The ABE engine fails during its initialization, right after opening ntdll, silently. Without the debug build, this error would have remained invisible.

3.5 Analyzing the ABE source code

The launch sequence (src/internal/thread.rs) follows this order:

```
AbSpawnActiveBreachThread()
1. AbEnsureSyscallTableInit()
- map_ntdll_image() <- "Opening ntdll.dll" (seen in logs)
- ex_syscalls() <- SSN extraction from the on-disk copy
- alloc_syscall_table_page() <- AbVirtualAlloc(): ALLOCATION VIA THE SYSCALL
PROLOGUE OF THE IN-MEMORY NTDLL
2. AbResolveNtdllSyscallPrologue("NtCreateThreadEx")
3. NtCreateThreadEx via this prologue <- dispatcher startup
```

And the official comment from src/internal/vm.rs, which decides the fate of the tool:

```
// If the export stub is fully hooked (no intact syscall prologue), these wrappers
// fail (return null/false). That is intentional: there is no safe
// "jump to entry" destination.
```

ABE is therefore fail closed by design: if the syscall prologue of the loaded ntdll is hooked or missing, the tool refuses to start. With the release build masking every error as 0, this decision manifests as an infinite hang with no message.

3.6 Dumping the ntdll stubs

A healthy Windows 11 syscall stub starts with 4C 8B D1 B8 (mov r10, rcx ; mov eax, SSN). A stub starting with E9 (jmp rel32) means an inline hook placed by a third party. Direct read from PowerShell (ReadProcessMemory):

ntdll function	First bytes	State
NtCreateThreadEx	4C 8B D1 B8 C9 00 00 00 ...	intact
NtQuerySystemInformation	4C 8B D1 B8 36 00 00 00 ...	intact
NtAllocateVirtualMemory	E9 A3 F2 FE BF 00 00 00 ...	hooked
NtClose	E9 63 F3 FE BF CC CC CC	hooked

The hook is selective: it touches the memory-management and handle APIs, the ones an EDR watches, and leaves the other two intact. Microsoft never ships syscall stubs starting with a jmp: these are third-party hooks, present in every process.

3.7 Tracing the hook chain to SophosED.dll

The hook jumps to an anonymous executable page located roughly 1 GB below ntdll (an atypical allocation pattern, non-Microsoft). Full trace of the chain, jmp by jmp:

#	Step	Detail
1	hooked ntdll stub	NtAllocateVirtualMemory - beginning replaced with E9
2	anonymous trampoline	private executable page, ntdll - 1 GB; FF 25 + E9 + CC padding
3	intermediate gate	indirect jmp FF 25 whose inline pointer is 0x7FFE4CA2F710
4	filtering code	0x7FFE4CA2F710 = SophosED.dll base + 0x5F710
5	relocated original stub	in the trampoline page: 4C 8B D1 B8 18 00 = mov r10, rcx ; mov eax, 0x18 (SSN 24 = NtAllocateVirtualMemory)
6	Windows kernel	syscall execution then return to ntdll+5

Step 4 is the attribution proof: the hook's filtering code resides inside SophosED.dll. Step 5 shows the complete mechanism: Sophos relocates the original syscall stub into its trampoline page. This pattern (direct jmp on the stub, indirect jmp into the EDR DLL, stub relocated in anonymous memory) matches the public Signal Labs analysis of Sophos hooking [1], whose study example is NtProtectVirtualMemory (hook pointing to hmpalert.dll): same mechanism, same conclusion. Our observation on NtAllocateVirtualMemory and the attribution to SophosED.dll are our own measurements.

No Sophos alert: expected behavior

These hooks are permanent surveillance: SophosED.dll is injected into every process and hooks continuously, regardless of the running tool. Nothing triggers at test time, so nothing in Event Viewer or the console. Sophos never detected ABE at any point: ABE tripped over permanent hooks, not over an alert.

3.8 A/B test: uninstalling Sophos

To confirm causality, Sophos was uninstalled and the machine rebooted. Same PoC, same DLL (debug build). Result: ABE initialization completes and the syscall executes:

```
[*] Launching ActiveBreach...
[AB:MAPPER][TID:9036] Opening C:\Windows\System32\ntdll.dll
[AB:EXPORTS][TID:9036] SYSCALL_TABLE initialized @0x1fd98450000 (entries=488)
[AB:EXPORTS][TID:9036] SYSCALL_TABLE set to READONLY
[AB:EXPORTS][TID:9036] SYSCALL_TABLE hash = 0x17E627DBB9C53E22
[+] activebreach_launch() returns: 0
[*] Calling NtQuerySystemInformation via ABE...
[AB:DISPATCH][TID:6196] dispatcher thread started @0x7fff9039ac80 (TID=6196)
[AB:STUB][TID:6196] stub pool ready
[+] SUCCESS! Syscall executed without detection.
ReturnLength = 256056 bytes
First buffer byte: 0x50
[*] PoC complete.
```

Every step missing under Sophos is present: table of 488 extracted SSNs, switch to READONLY, integrity hash, dispatcher thread, stub pool. The syscall returns a 256 KB process list. Only PoC change: buffer increased from 4 KB to 1 MB, the kernel initially returning 0xC0000004 (buffer too small).

4. Root cause: the complete mechanism

#	Actor	Action
1	Sophos (passive)	SophosED permanently hooks the NtAllocateVirtualMemory and NtClose stubs (trampoline + filtering in anonymous memory)
2	ABE	Attempts to allocate its syscall table via the syscall prologue of the loaded ntdll; it finds the stub modified
3	ABE	By design (fail closed, official code comment), ABE gives up: launch() returns an error
4	ABE (release build)	In release builds, the error is returned as 0, indistinguishable from success
5	PoC	The PoC, which does not check the return code, calls ab_call; no dispatcher exists to handle the request
6	ABE	ab_call waits for the dispatcher on a G_READY variable via WaitOnAddress(INFINITE): infinite hang, 0% CPU

```
SophosED: permanent hooks (NtAllocateVirtualMemory / NtClose)
|
v
ABE: table allocation via ntdll prologue
|
v
Hooked stub detected: ABE gives up (fail closed)
|
v
Error masked as 0 (release build)
|
v
PoC calls ab_call with no dispatcher running
|
v
Infinite hang on WaitOnAddress (0% CPU)
```

Precise formulation

Sophos did not actively detect or block ABE (no EDR-side decision, no alert). Sophos created the condition (permanent hooks); ABE blocked itself by enforcing its own integrity policy. Functionally, Sophos renders ABE inoperable; mechanically, the refusal comes from ABE's own code.

5. Conclusion

Phase 1: on a host protected by Sophos Endpoint, ABE v0.2.0 (ntdll_backend mode) does not work and bypasses nothing: its initialization fails and manifests as an infinite hang, the real error being masked as 0 by release builds.

The cause was attributed conclusively: SophosED user-mode hooks on NtAllocateVirtualMemory / NtClose, with attribution proof (filtering code located at SophosED.dll+0x5F710, relocated original stub observed, pattern consistent with the public Signal Labs research [1]).

Without an EDR, the tool works as designed: SSNs extracted dynamically (488 entries), fresh stubs in private memory, syscalls executed directly. User-mode layer bypass is real as soon as initialization succeeds.

Cross-cutting lesson: error codes masked in release and the missing launch() return-code check made the diagnosis significantly longer. Any evaluated tool must be assessed in debug builds with strict return-code verification.

Update after the bypass phases (annexes C to E)

The complete bypass was ultimately validated with Sophos active. The user-mode layers (SophosED hooks and the HitmanPro.Alert SysCallX64 mitigation) delay an attacker by a few hours of adaptation; they do not stop them. No alert was raised in the Sophos console during the complete sequence. Effective detection must rely on kernel telemetry.

6. Recommendations (detection and hardening)

These recommendations are confirmed by the bypass validated in annex E: the complete sequence (bypassed initialization + executed syscall) produced no alert in the Sophos console.

6.1 Detecting such an evasion (independent of user-mode hooks)

- **Stack-walking within kernel callbacks**: callbacks (memory allocation, thread creation) must verify the return address. A return address pointing into an unbacked private RX region is the signature of direct syscalls; a direct ntdll prologue call from a third-party module (without going through kernelbase/kernel32) is likewise anomalous. Undetectable from user mode, visible only from the kernel.
- **Memory scanning for syscall stubs**: the signature 4C 8B D1 B8 xx 00 00 00 0F 05 C3 outside ntdll is a homemade syscall stub. ABE creates dozens of them in its pool; no legitimate software exhibits this behavior. Simple detection, very low false-positive rate.
- **Integrity of the EDR's own hooks**: the EDR knows the expected byte (E9) at the start of the stubs it hooks. Any restoration or replacement (unhooking) must trigger an immediate alert. Simple fix, strong deterrent effect.
- **Correlate kernel telemetry** (ETW-TI, Ob/Ps callbacks) into visible alerts: a striking finding of this test is that none of this telemetry was turned into a console alert during the bypass. The gap is not only technical, it is also one of correlation.

6.2 Prevention

- **HVCI / VBS** (Virtualization-Based Security) and Control Flow Guard make the creation and execution of private stubs significantly more expensive for the attacker.
- **Behavioral rule**: alert on the sequence DLL load, private RX allocation, ntdll prologue call outside the normal API path. Signature observed in ABE, generalizable to evasion tools.

Governance

Keep these tests on the lab VM, retain the snapshots and journal exports. The deliverables (hang, attribution, A/B, validated bypass) form a complete, reproducible dossier.

Annex A - Key commands

Compiling ABE (tested modes):

```
cd C:\Tools\ActiveBreach-Engine\SDK\Rust
cargo build --release --features ntdll_backend # release (errors masked as 0)
cargo build --features ntdll_backend # debug (logs + real error codes)
cargo build # debug, secure backend by default
```

Hook verification (re-run in every new PowerShell session):

```
Add-Type @"..."@ # Mem class: GetModuleHandle / GetProcAddress / ReadProcessMemory
$nt = [Mem]::GetModuleHandle("ntdll.dll")
foreach ($f in @("NtCreateThreadEx","NtQuerySystemInformation","NtAllocateVirtualMemory","NtClose"))
{
$p = [Mem]::GetProcAddress($nt, $f)
$b = [Mem]::Read($p, 8)
Write-Host ("{0,-30}{1}" -f $f, (($b | ForEach-Object { $_.ToString("X2") }) -join ""))
}
# 4C 8B D1 B8 ... = healthy stub | E9 ... = inline hook
```

Trampoline tracing (pointer resolution and attribution):

```
$rel = [BitConverter]::ToInt32($b, 1)
$tgt = [int64]$p + 5 + $rel
# observed FF 25 target: 0x7FFE4CA2F710 = SophosED.dll base + 0x5F710
```

A/B test:

```
# 1) observe the failure (VM with Sophos)
# 2) uninstall Sophos + reboot
# 3) dump the stubs: 4C 8B D1 B8 everywhere (hooks gone)
# 4) re-run the PoC
```

Annex B - Excerpts: PoC and PowerShell scripts

Main PoC fix (return-code check and properly sized buffer):

```
let ret = launch();
if ret != 0 {
println!("[-] launch() FAILED: {:#x} - aborting before ab_call.", ret);
return;
}
let mut buffer = vec![0u8; 1024 * 1024]; // 1 MB (SystemProcessInformation)
```

Phase 1 result (VM without EDR):

```
[+] SUCCESS! Syscall executed without detection.
ReturnLength = 256056 bytes
First buffer byte: 0x50
```

Annex C - Evasion scenarios (phase 2)

Three approaches to overcome ABE's fail-closed behavior with Sophos active, in order of simplicity. Common objective: succeed at initialization, execute the syscalls, measure what the EDR observes. Always in debug builds, always on the lab VM.

C.1 Approach A (selected): patch ABE, VirtualAlloc fallback

Initialization fails because AbVirtualAlloc (src/internal/vm.rs) requires an intact ntdll prologue to allocate the syscall table. Patch: when resolution fails (hooked stub), fall back to the classic Win32 API instead of the direct syscall.

```
// src/internal/vm.rs - fallback inside AbVirtualAlloc
use windows::Win32::System::Memory::{VirtualAlloc, MEM_COMMIT, MEM_RESERVE, PAGE_PROTECTION_FLAGS};
let p = unsafe { VirtualAlloc(None, size, MEM_COMMIT | MEM_RESERVE, PAGE_PROTECTION_FLAGS(protect))
};
if p.is_null() { return core::ptr::null_mut(); }
```

The initial allocation becomes visible again (API and kernel callbacks), which is intended: we measure whether Sophos relies on its user-mode hooks or on its kernel layer and correlation engine.

C.2 Approach B: homemade stub from the clean ntdll copy

ABE already maps a clean on-disk copy of ntdll to extract the SSNs. Stronger variant: build an internal syscall stub from that clean image and use it for NtAllocateVirtualMemory, making the table allocation invisible to the hooks.

C.3 Approach C: reusing Sophos's relocated stub (read-only)

Documented technique (Signal Labs [1]) and demonstrated by the phase 1 attribution scripts: the Sophos trampoline contains the original stub (4C 8B D1 B8 18 00, SSN 0x18). Following it and calling it requires no memory write, hence no memory-protection alert risk.

C.4 Measurements at every step

- Tool side: debug build, init must show SYSCALL_TABLE initialized (488), dispatcher, stub pool, then SUCCESS.
- EDR side: Sophos Central console, Event Viewer, System Informer. Systematically record observed vs. not observed per layer.
- Scope: these scenarios bypass the user-mode layer. Sophos kernel callbacks (Ob/Ps) and ETW-TI keep seeing allocations and thread creations.

Annex D - Blocked by the anti-exploit

D.1 vm.rs patch v2: the neighboring stub trap

Patch v1 (fallback only when resolution returns None) proved insufficient. Cause: Sophos only replaces 5 bytes of the hooked stub (4C 8B D1 B8 xx). The 96-byte scan of resolve_ntdll_prologue then overflows onto the neighboring, intact stub, whose canonical prologue is found: resolution succeeds at the wrong address, the executed syscall is the neighboring function's, and the call fails without triggering the fallback. Patch v2 fixes this: the fallback also triggers when the prologue call fails, detected by checking the NTSTATUS AND the non-null base.

D.2 Result under Sophos: initialization bypassed

With patch v2 and Sophos active (four verified hooked stubs: NtAllocateVirtualMemory, NtProtectVirtualMemory, NtFreeVirtualMemory, NtClose), the entire initialization passes, with a table hash identical to the clean machine:

```
[AB:EXPORTS] SYSCALL_TABLE initialized @0x1b94bc90000 (entries=488)
[AB:EXPORTS] SYSCALL_TABLE set to READONLY
[AB:EXPORTS] SYSCALL_TABLE hash = 0x17E627DBB9C53E22 <- identical to the clean machine
[+] activebreach_launch() returns: 0
[AB:DISPATCH] dispatcher thread started (TID=3172)
[AB:STUB] stub pool ready
```

D.3 The next blocker: execution killed by the anti-exploit

The process dies silently at the moment of the syscall execution. The exit code confirms an access violation (0xC0000005). Log correlation attributes it unambiguously to the anti-exploit mitigation:

Timestamp	Source	Detail
16:27:28	HitmanPro.Alert (ID 911)	SysCall mitigation, SysCallX64 policy, PID 11752, application abe_poc.exe
16:27:29	Application Error (ID 1000)	abe_poc.exe, PID 0x2DE8 (= 11752), exception 0xC0000005, faulting module "unknown", fault offset 0xbfe1ff530 (anonymous region of the mitigation)

Same process, one second apart: the HitmanPro.Alert SysCall mitigation detects and kills the syscall execution.

D.4 Why HMPA sees the syscall (SysCallX64 policy analysis)

Even in ntdll_backend mode, ABE does not call the ntdll prologue directly: the dispatcher writes a jump stub (write_jmp64_stub) into its own private RX page and calls that stub. At the moment of the syscall instruction, the return address on the stack points into an unbacked anonymous region, the exact signature of the SysCallX64 policy. The anti-exploit enforces this criterion.

D.5 Intermediate summary

Defense layer	Result vs. patched ABE	Finding
SophosED user-mode hooks (init)	Bypassed (fallback patch v2)	Insufficient alone: ABE initializes normally
HMPA anti-exploit (execution)	Blocking (SysCallX64 kills the process)	Detects syscall execution from private memory
Kernel callbacks / ETW-TI	Final observer	Sees allocations and thread creations no matter what

Annex E - Complete bypass validated under active Sophos (phase 3)

Final phase: not just initialize ABE under Sophos, but execute a syscall end to end. Result: success. The required modifications illustrate why user-mode heuristics alone do not hold.

E.1 The complete bypass chain (3 modifications)

#	File	Modification	Bypasses
1	src/internal/vm.rs	Win32 fallback when the prologue is missing OR when the call fails (neighboring stub trap). NTSTATUS and non-null base check.	User-mode hooks (init)
2	src/internal/exports.rs	Prologue scanner made tolerant of the optional test/jne block of modern Win11 stubs (F6 04 25 ... 01 / 75 xx) between mov eax and syscall. Without this fix, the prologue cache stays empty and the dispatcher always falls back to the private stub.	Prologue cache
3	src/internal/dispatch.rs	DIRECT call of the ntdll prologue from the dispatcher (instead of a jump stub in private memory): the syscall physically executes in the ntdll .text and the return address is in the .text of a real module.	SysCallX64 mitigation

E.2 Result: syscall executed with Sophos active

Hook state at test time: four of six verified stubs hooked. Complete output:

```
[*] Launching ActiveBreach...
[AB:MAPPER] Opening C:\Windows\System32\ntdll.dll
[AB:EXPORTS] SYSCALL_TABLE initialized @0x240c4060000 (entries=488)
[AB:EXPORTS] SYSCALL_TABLE set to READONLY
[AB:EXPORTS] SYSCALL_TABLE hash = 0x17E627DBB9C53E22
[+] activebreach_launch() returns: 0
[AB:DISPATCH] dispatcher thread started (TID=4016)
[*] Calling NtQuerySystemInformation via ABE...
[AB:STUB] stub pool ready
[AB:DISPATCH][L3] direct ntdll prologue call
[+] SUCCESS! Syscall executed without detection.
ReturnLength = 288280 bytes
First buffer byte: 0x50
[*] PoC complete.
```

The kernel returned 288 KB of process list. The complete syscall execution took place while the SophosED hooks were active and HitmanPro.Alert was running. An independent check (embedded string in the DLL) confirms the patch was in the binary.

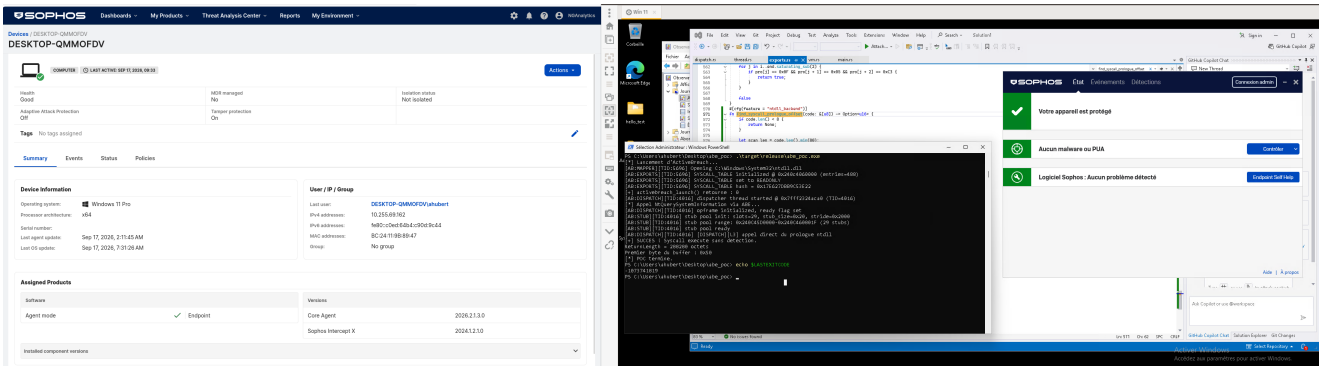


Figure 1 - Execution proof, 09/17/2026 (09:33-09:37): Sophos Central console with no alert (left) and complete PoC output with the L3 patch active (right).

E.3 What this demonstration establishes

Layer	Verdict	Evidence
SophosED user-mode hooks	Bypassed	Full init under hooks (annexes C and D)
HMPA SysCallX64 anti-exploit	Bypassed	Heuristic based on the syscall RIP: the direct prologue call executes it in the ntdll .text. Return-address spoofing alone was insufficient (process killed, annex D); the direct call passes.
Console / alerts	No alert	Nothing raised in the Sophos Central console during the entire sequence (bypassed init, executed syscall, process exit)
Kernel callbacks / ETW-TI	Only remaining barrier	The syscall really executes: the kernel layer observes it. Nothing was correlated or alerted on the console side during the test.

Rigor note

The process exited with code 0xC0000005 after printing "PoC complete", a post-execution reaction that remains unqualified (no event raised in the console). It does not affect the result: the syscall had executed and its response processed.

E.4 Updated recommendations

- **Priority 1, mandatory kernel detection**: both Sophos user-mode layers can be bypassed with minor modifications to a public tool. Detection via stack-walking in callbacks (spotting direct ntdll prologue calls from third-party modules), ETW-TI fed into the engine, and correlation into a visible alert: the test shows that existing telemetry was not turned into any alert.
- **Priority 2, hardware prevention**: HVCI/VBS and Control Flow Guard make the creation and execution of private stubs significantly more expensive.
- **Priority 3, behavioral rule**: alert on the sequence DLL load, private RX allocation, ntdll prologue call outside the normal API path. Signature observed in ABE, generalizable.

Final finding

The experiment proves that "EDR installed" is worth nothing without "kernel telemetry exploited and correlated into alerts". Hooks and user-mode anti-exploit delay an attacker (multiple attempts, multiple patches, multiple hours); they do not stop them.

End of report. Related material: Proxmox VM snapshots, journal exports (Event Viewer) and test DLLs archived in the project folder.

References

[1] Signal Labs. "In-Memory Disassembly for EDR/AV Unhooking" - analysis of Sophos hooks (study example: NtProtectVirtualMemory, hook to hmpalert.dll) and locating the original relocated syscall stub via in-memory disassembly. April 2023. https://signal-labs.com/analysis-of-edr-hooks-bypasses-amp-our-rust-sample/